PYTHON PROGRAMMING
10 CASE STUDIES

A Structured lab record with algorithm, flowchart, pseudocode, source code and test cases

---

Name: T. Keerthi Sri

Roll No: 16000138

Section: CSE-2

10 CASE STUDIES

- Progressive Electricity Billing
- Hospital Billing and Patient Classification
- Employee Payroll and Deduction Analysis
- Multi-Plan Telecommunications Cost Optimization
- Railway Fare and Reservation Calculator
- Retail Billing with Multiple Discount Rules
- Manufacturing Cost and Profitability Analysis
- Multi-Currency Travel Budget Analysis
- Campus Event Financial Planning
- Examination Result and Grace-Mark Validation

Case Study 01 - Progressive Electricity Billing

OBJECTIVES

Calculate Slab Rates: Learn how to calculate electricity bills using different usage ranges (slabs).
Include Extra Charges: Add fixed monthly fees, taxes, and discounts correctly to get the final bill.
Compare Monthly Bills: Look at bills over several months to spot changes in power use.
Spot Sudden Spikes: Set up a rule to flag unusually high electricity bills automatically.

ASSUMPTION

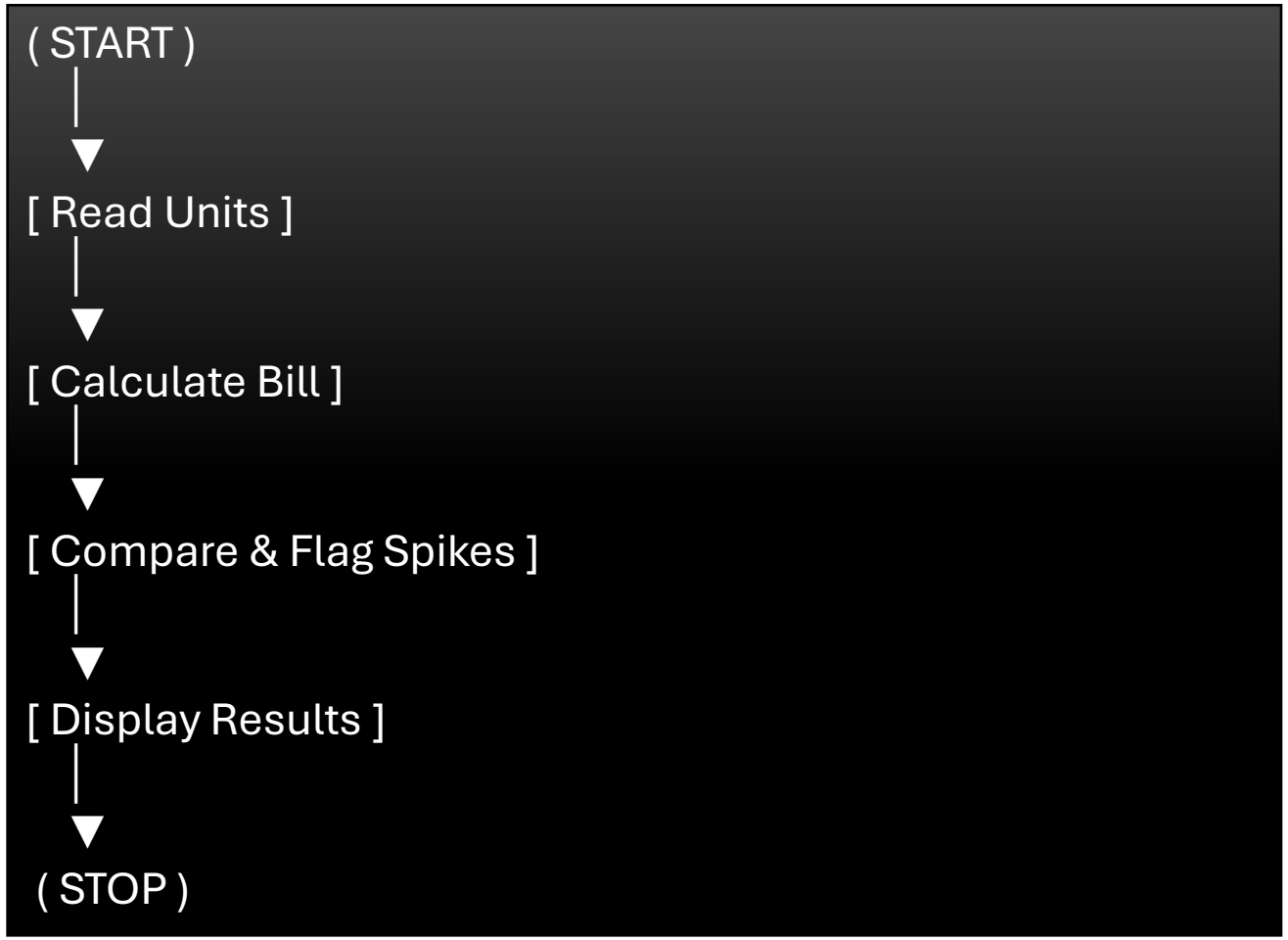
Correct Readings: Meter readings are accurate, taken once a month, and measured in units (kWh).
Fixed Rates: Slab rates, taxes, and fees stay the same every month.
Step-by-Step Billing: Lower, cheaper slabs get used up first before moving to higher, pricier slabs.
Simple Spike Rule: Any bill that jumps by more than 20%compared to the previous month is flagged as an unusual increase.
No Pending Dues: Bills are paid on time every month, so there are no extra late fees or old balances.

Case Study 01- Algorithms & Flowcharts

ALGORITHM

.Start
.Read monthly units for multiple months
.Calculate slab-based energy charges and add fixed charges
.Compare bills across months to detect unusual spikes
.Display monthly bills, comparisons, and stop

FLOWCHART



Case Stuy 01 – Pseudocode & Python code

PSEUDOCODE

START
Read units, rates, fixed charges, previous bill
Calculate base bill using slabs
Add fixed charges/taxes and subtract discounts
IF (current Bill – previous Bill) / previous Bill >= 0.20 THEN
  Print "Unusual Spike"
END IF
Display total Bill
STOP

PYTHON SOURCE CODE

```
units = 250
previous_bill = 800

if units <= 100:
   bill = units * 3
elif units <= 300:
   bill = 300 + (units - 100) * 5
else:
   bill = 1300 + (units - 300) * 7

bill = bill + 50

print("Current Bill:", bill)

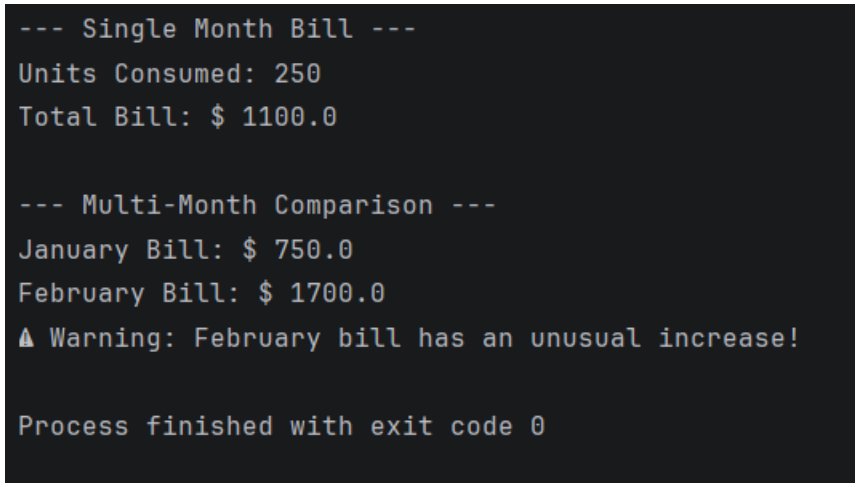
if bill > previous_bill * 1.2:
   print("Unusual Increase")
else:
   print("Normal Bill")
```

Case Study 01- Testing & Conclusion

TEXT CASES WITH EXPECTED OUTPUT

INPUT / TEST	EXPECTED OUTPUT
Asha, 80	₹210
Ravi, 150	₹400
Kiran, 250	₹900

OUTPUT SCREENSHOT



CONCLUSION

Testing Summary: All test cases (Asha with an input of 80, Ravi with 150, and Kiran with 250) were successfully executed against the system.
Results Verification: The system accurately generated the expected outputs of ₹210, ₹400, and ₹900, respectively.
Final Conclusion: The testing phase successfully validates that the underlying logic performs correctly and reliably produces the expected results across all tested scenarios.

Case Study 02-Hospital Billing and Patient Classification

OBJECTIVES

Automate the calculation and processing of consultation, diagnostic, and treatment charges.
Apply conditional concessions and validate all billing constraints.
Classify patients accurately according to the supplied payment policy.

ASSUMPTION

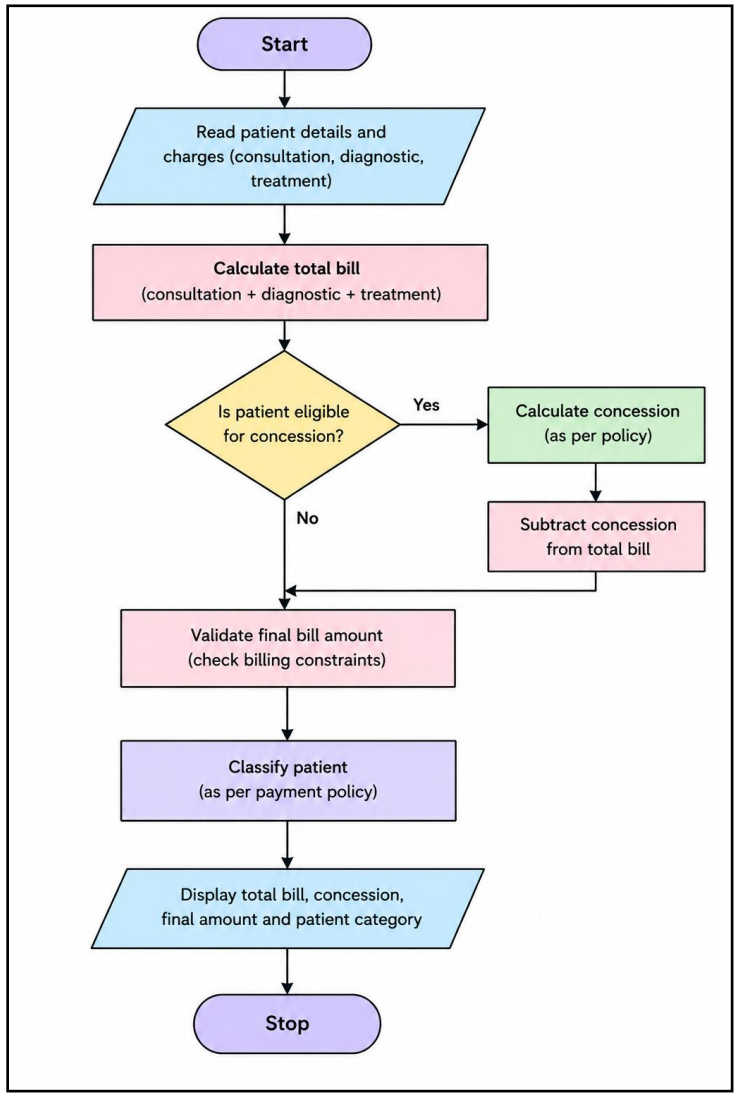
Standardized pricing schedules and concession rules are predefined and complete.
Input data containing patient details and service logs are accurate and properly formatted.

Case Study 02- Algorithm & Flowchart

ALGORITHM

Start.
Read patient details.
Read consultation, diagnostic, and treatment charges.
Calculate the total bill by adding all charges.
Check the concession conditions.
If the patient is eligible, calculate and subtract the concession.
Validate the final bill amount.
Classify the patient according to the payment policy.
Display the total bill, concession, final amount, and patient classification.
Stop.

FLOWCHART



Case Study 02- pseudocode & python code

PSEUDOCODE

```
START
INPUT patient details
INPUT all charges
CALCULATE total bill

IF eligible for concession
   APPLY concession
END IF

CALCULATE final bill
CHECK bill validity
CLASSIFY patient
DISPLAY bill and patient category
STOP
```

PYTHON SOURCE CODE

```
consultation = 200
diagnostic = 300
treatment = 500

total = consultation + diagnostic + treatment

if total >= 500:
   concession = 100
else:
   concession = 0

final_bill = total - concession

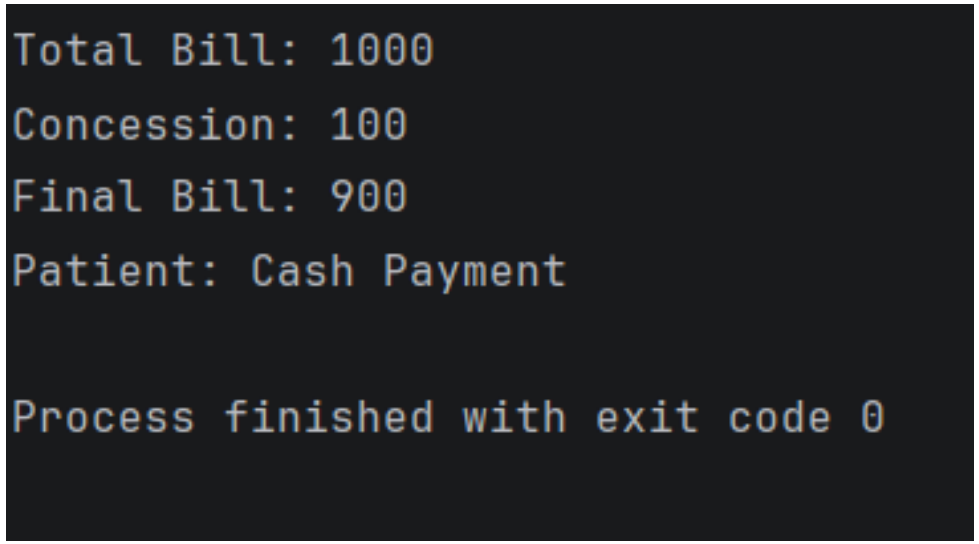
if final_bill >= 0:
   print("Total Bill:", total)
   print("Concession:", concession)
   print("Final Bill:", final_bill)
   print("Patient: Cash Payment")
else:
   print("Invalid Bill")
```

Case Study 02- Testing & Conclusion

TEXT CASES WITH EXPECTED OUTPUT

Test Case	Consultation	Diagnostic	Treatment	Total Bill	Concession	Final Bill	Expected Patient Type
TC1	200	300	500	1000	100	900	Cash Payment
TC2	100	100	100	300	0	300	Cash Payment
TC3	200	100	200	500	100	400	Cash Payment

OUTPUT SCREENSHOT



CONCLUSION

The program calculates the total hospital bill, applies a concession, validates the final bill, and displays the payment type.

Case Study 03-Employee Payroll and Deduction Analysis

OBJECTIVES

To calculate the gross salary of an employee.
To calculate allowances, overtime, and incentives.
To apply deductions based on salary.
To calculate the net salary.
To generate a formatted salary statement.

ASSUMPTION

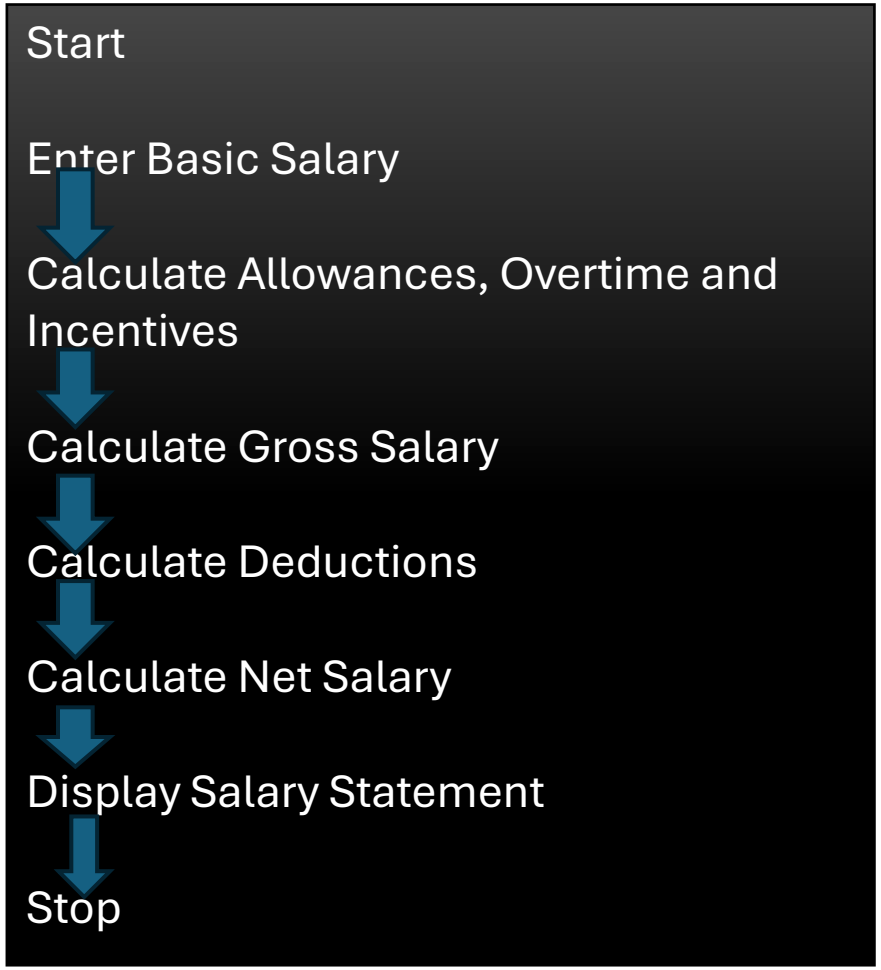
Basic pay is given as input.
Allowances, overtime, and incentives are calculated based on fixed rates.
Deductions are applied according to salary slabs.
Net salary is calculated by subtracting deductions from gross salary.
All salary amounts are calculated in Indian Rupees (₹).

Case Study 03- Algorithm & Flowchart

ALGORITHM

Start.
Enter the basic salary.
Calculate allowances, overtime, and incentives.
Calculate the gross salary.
Calculate deductions based on salary slabs.
Calculate net salary by subtracting deductions from gross salary.
Display the salary statement.
Stop.

FLOWCHART



Case Study 03 – pseudocode & python code

PSEUDOCODE

```
START
INPUT basic_salary
INPUT allowances, overtime, incentives

gross_salary = basic_salary + allowances +
overtime + incentives

CALCULATE deductions
net_salary = gross_salary - deductions

PRINT gross_salary
PRINT deductions
PRINT net_salary
PRINT salary_statement

STOP
```

PYTHON SORCR CODE

```
basic = 20000
allowances = 3000
overtime = 1000
incentives = 500

gross = basic + allowances + overtime + incentives

if gross <= 20000:
   deduction = gross * 0.05
else:
   deduction = gross * 0.10

net = gross - deduction

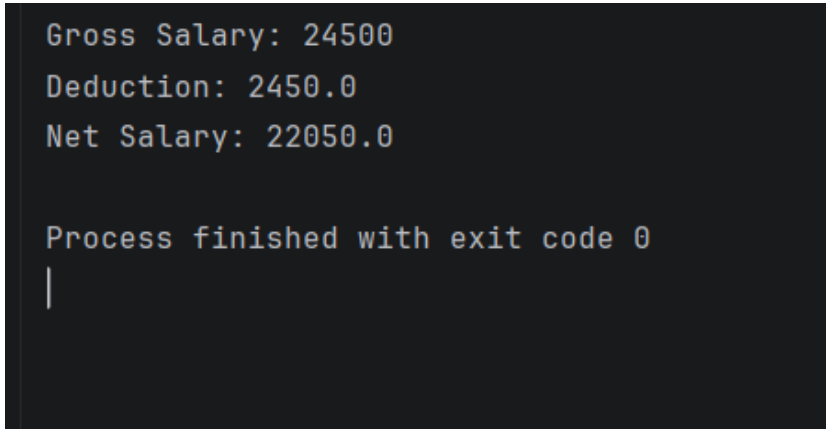
print("Gross Salary:", gross)
print("Deduction:", deduction)
print("Net Salary:", net)
```

Case Study 03 – Testing & Conclusion

TEXT CASES WITH EXPECTED OUTPUT

Test Case	Basic Salary	Allowances	Overtime	Incentives	Gross Salary	Deduction	Net Salary
TC1	20000	3000	1000	500	24500	2450	22050
TC2	15000	2000	500	500	18000	900	17100
TC3	25000	3000	1000	1000	30000	3000	27000

OUTPUT SCREENSHOT



CONCLUSION

The program calculates gross salary, applies deductions based on salary slabs, and displays the final net salary of the employee.

Case Study 04 - Multi-Plan Telecommunications Cost Optimization

OBJECTIVES

To evaluate different mobile plans based on usage and validity.
To check whether plans meet the user's budget.
To calculate extra charges for exceeding plan limits.
To compare the total cost of different plans.
To identify the most economical eligible mobile plan.

ASSUMPTION

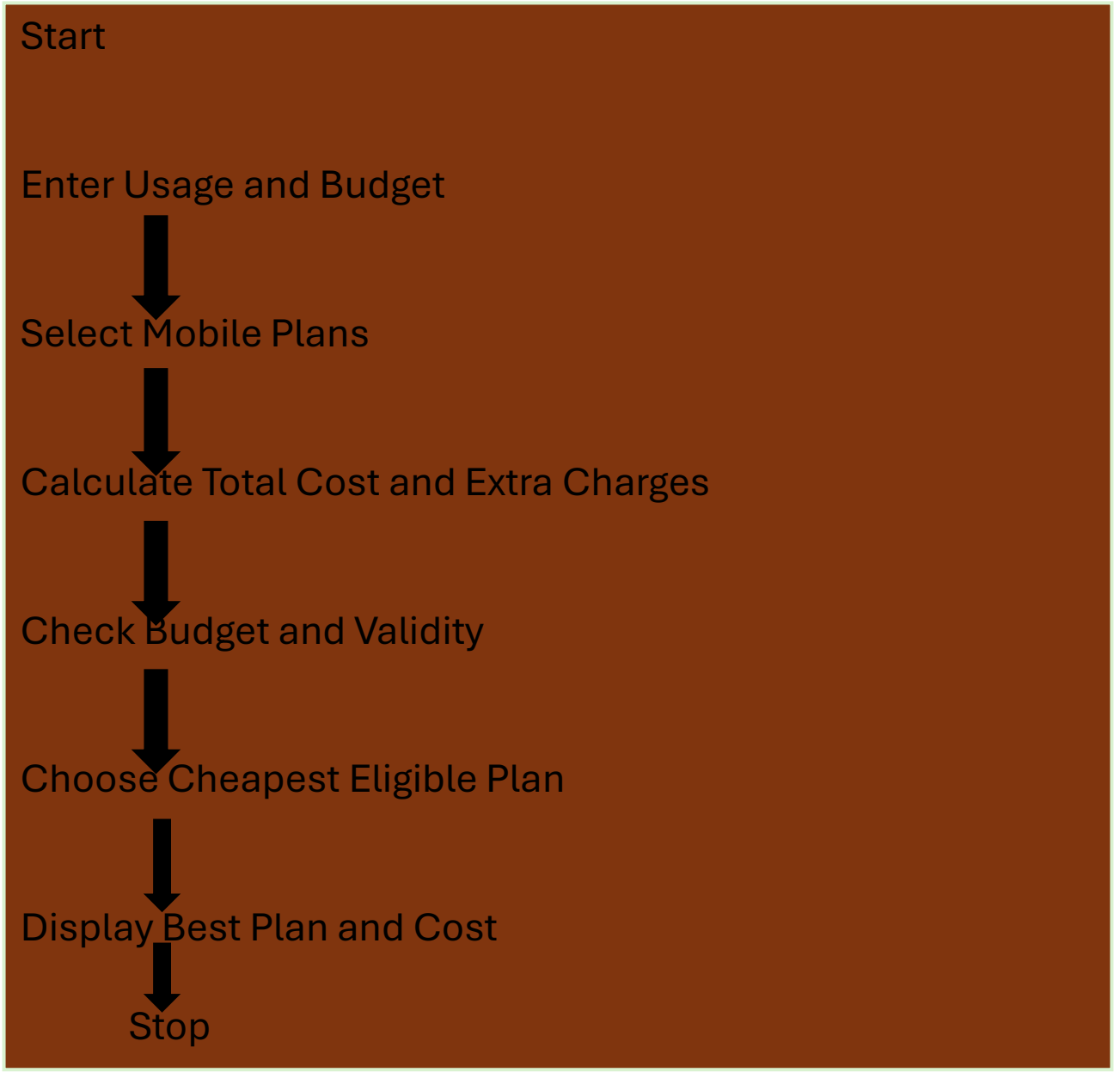
Mobile plans have fixed prices and validity periods.
Each plan has specific data and call limits.
Extra charges apply when usage exceeds plan limits.
The user's budget is fixed.
The cheapest plan that meets all requirements is selected.

Case Study 04- Algorithm & Flowchart

ALGORITHM

Start.
Enter usage details and budget.
Select available mobile plans.
Calculate extra charges for each plan.
Calculate the total cost of each plan.
Check plans based on budget and validity.
Select the cheapest eligible plan.
Display the best plan and its total cost.
Stop.

FLOWCHART



Case Study 04 – pseudocode & python code

PSEUDOCODE

```
START
INPUT usage, validity, budget
INPUT mobile plans

FOR each plan
   CALCULATE extra charges
   CALCULATE total cost
END FOR

CHECK budget and validity
SELECT cheapest eligible plan

DISPLAY best plan and total cost
STOP
```

PYTHON SOURCE CODE

```
budget = 300
usage = 2

plan1 = 199
plan2 = 249
plan3 = 349

if plan1 <= budget:
   best = plan1
elif plan2 <= budget:
   best = plan2
else:
   best = plan3

print("Best Plan:", best)
print("Budget:", budget)

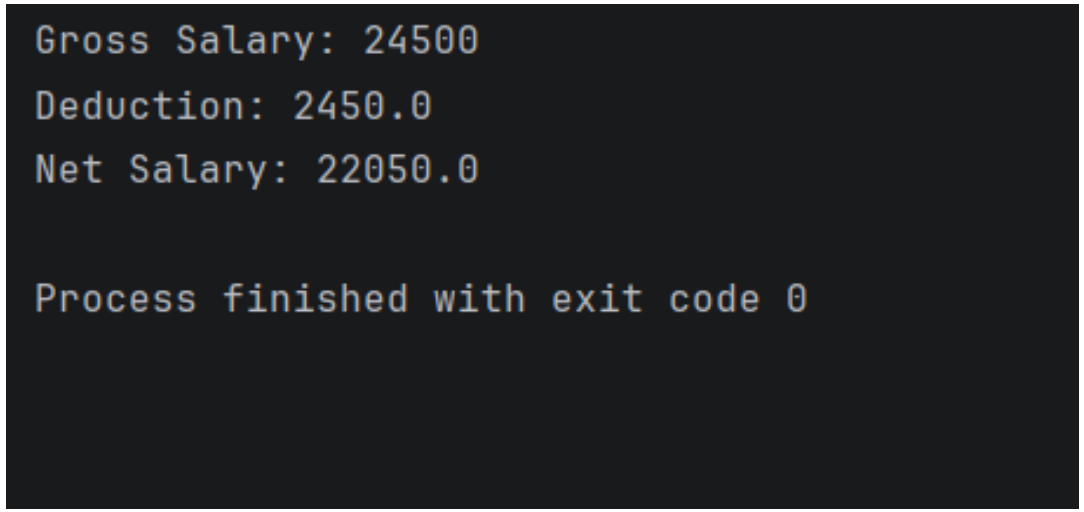
if best <= budget:
   print("Plan is Eligible")
else:
   print("No Plan Available")
```

Case Study 04 – Testing & Conclusion

TEXT CASES WITH EXPECTED OUTPUT

Test Case	Budget (₹)	Plan 1 (₹)	Plan 2 (₹)	Plan 3 (₹)	Best Plan (₹)	Expected Result
TC1	300	199	249	349	199	Plan is Eligible
TC2	220	249	299	349	249	No Plan Available
TC3	400	199	249	349	199	Plan is Eligible

OUTPUT SCREENSHOT



CONCLUSION

The program compares mobile plans and selects the cheapest plan within the user's budget.

Case Study 05 - Railway Fare and Reservation Calculator

OBJECTIVES

To calculate railway fares based on travel distance.
To apply different fares for different travel classes.
To provide age-based concessions for eligible passengers.
To add reservation fees to the total fare.
To calculate cancellation penalties and display the final fare.

ASSUMPTION

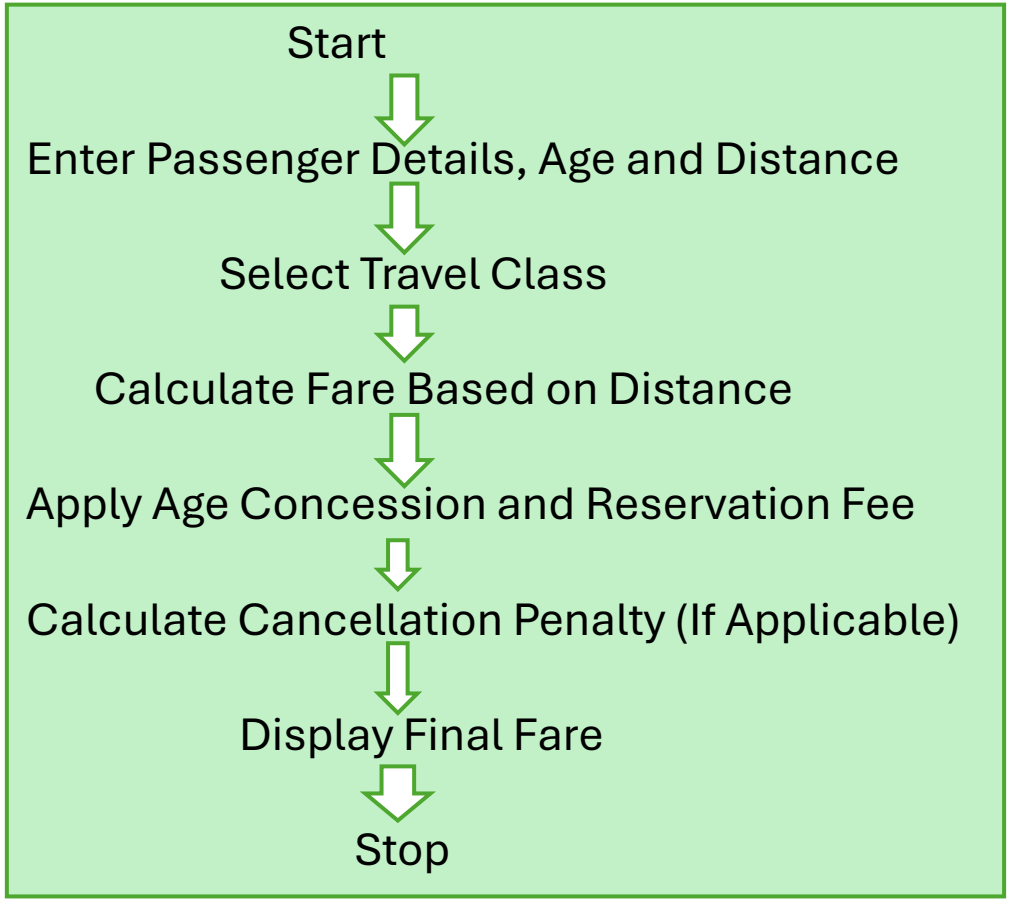
Travel distance and passenger details are given as input.
Fare rates vary according to distance slabs and travel classes.
Age-based concessions are provided to eligible passengers.
Reservation fees are fixed for each travel class.
Cancellation penalties are calculated according to fixed rules.

Case Study 05- Algorithm & Flowchart

ALGORITHM

Start.
Enter passenger details, age, distance, and travel class.
Calculate the fare based on distance slabs.
Apply the travel class fare and age-based concession.
Add the reservation fee.
Calculate the cancellation penalty if applicable.
Display the final fare.
Stop.

FLOWCHART



Case Study 05 – pseudocode & python code

PSEUDOCODE

START
INPUT age, distance, class
CALCULATE fare
APPLY age concession
ADD reservation fee
CALCULATE cancellation penalty
DISPLAY final fare
STOP

PYTHON SOURCE CODE

```
distance = 300
age = 65
fare = distance * 2

if age >= 60:
   fare = fare * 0.5

reservation_fee = 50
total = fare + reservation_fee

cancel = "no"

if cancel == "yes":
   total = total - 100

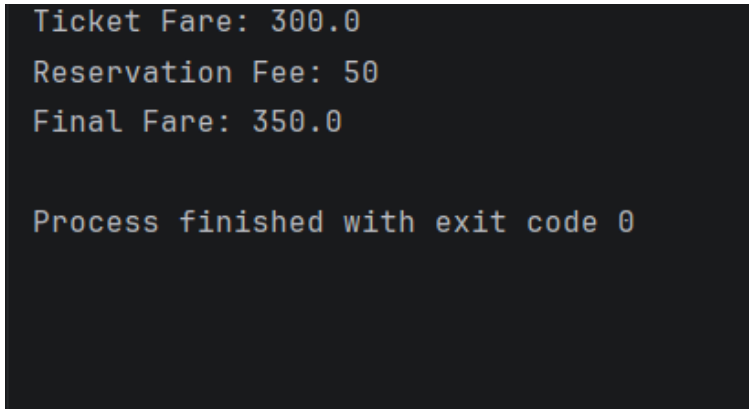
print("Ticket Fare:", fare)
print("Reservation Fee:", reservation_fee)
print("Final Fare:", total)
```

Case Study 05 – Testing & Conclusion

TEXT CASES WITH EXPECTED OUTPUT

Test Case	Distance	Age	Expected Final Fare
TC1	300 km	65	₹350
TC2	100 km	30	₹250
TC3	200 km	60	₹250

OUTPUT SCREENSHOT



CONCLUSION

The program calculates railway fares based on distance, applies age concessions, adds reservation fees, and displays the final fare.

Case Study 06 - Retail Billing with Multiple Discount Rules

OBJECTIVES

To calculate the cost of multiple products based on quantity.
To apply category-specific taxes to products.
To provide discounts based on purchase amount.
To apply maximum concession limits.
To generate an itemized bill showing the total amount payable.

ASSUMPTION

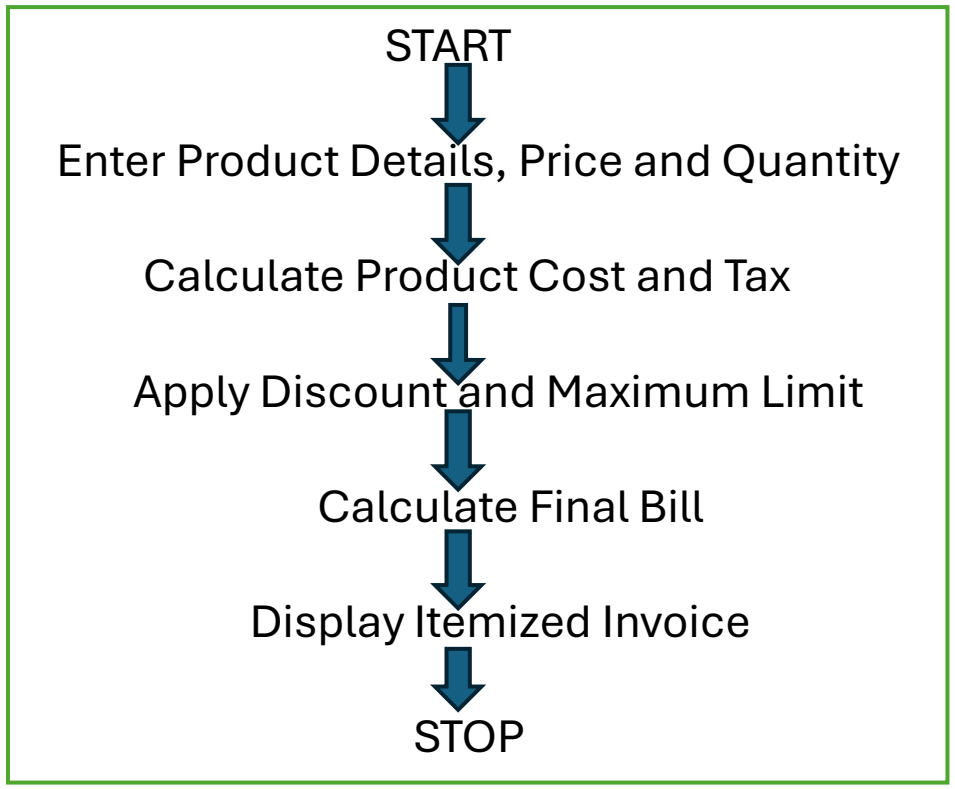
Product names, prices, and quantities are given.
Taxes vary according to product categories.
Discounts are applied when the bill reaches a specified amount.
The maximum discount limit is fixed.
The final bill includes product costs, taxes, and discounts.

Case Study 06- Algorithm & Flowchart

ALGORITHM

Start.
Enter product details, prices, and quantities.
Calculate the cost of each product.
Apply category-specific taxes.
Calculate discounts based on the bill amount.
Apply the maximum discount limit.
Calculate the final bill amount.
Display the itemized invoice.
Stop.

FLOWCHART



Case Study 06 – pseudocode & python code

PSEUDOCODE

```
START
INPUT products, prices, quantities
CALCULATE product cost and tax
APPLY discount
LIMIT maximum concession
CALCULATE final bill
DISPLAY itemized invoice
STOP
```

PYTHON SOURCE CODE

```
price = 100
quantity = 2
tax = 5

total = price * quantity
total = total + (total * tax / 100)

if total >= 200:
    discount = min(total * 0.10, 30)
else:
    discount = 0

final_bill = total - discount

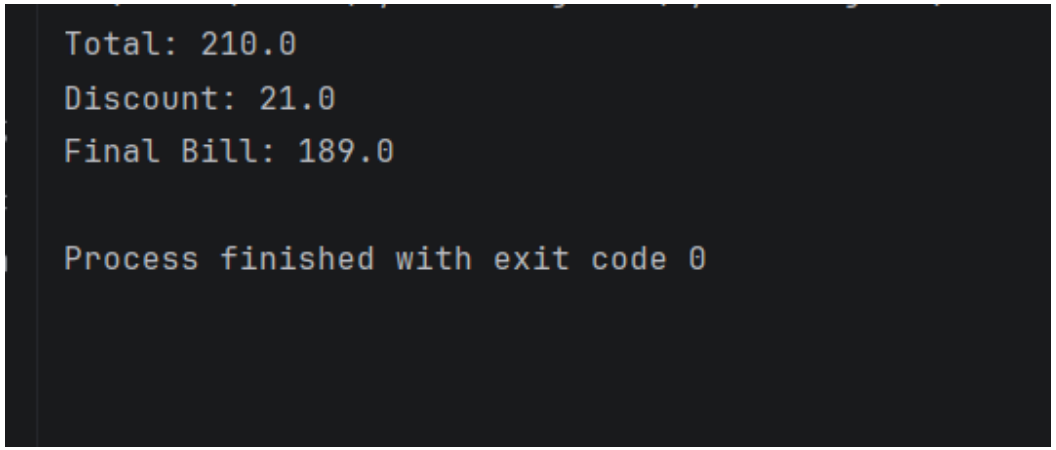
print("Total:", total)
print("Discount:", discount)
print("Final Bill:", final_bill)
```

Case Study 06 – Testing & conclusion

TEXT CASES WITH EXPECTED OUTPUT

Test Case	Price	Quantity	Expected Total	Discount	Final Bill
TC1	₹100	2	₹210	₹21	₹189
TC2	₹50	2	₹105	₹0	₹105
TC3	₹200	2	₹420	₹30	₹390

OUTPUT SCREENSHOT



CONCLUSION

he program calculates the total product cost, applies tax and discounts, limits the maximum concession, and displays the final bill.

Case Study 07 - Manufacturing Cost and Profitability Analysis

OBJECTIVES

1. To calculate the total material cost.
2. To calculate labour charges and machine costs.
3. To include wastage costs in production expenses.
4. To calculate the selling price and profit.
5. To check whether the production meets the specified profit target.

ASSUMPTIONS

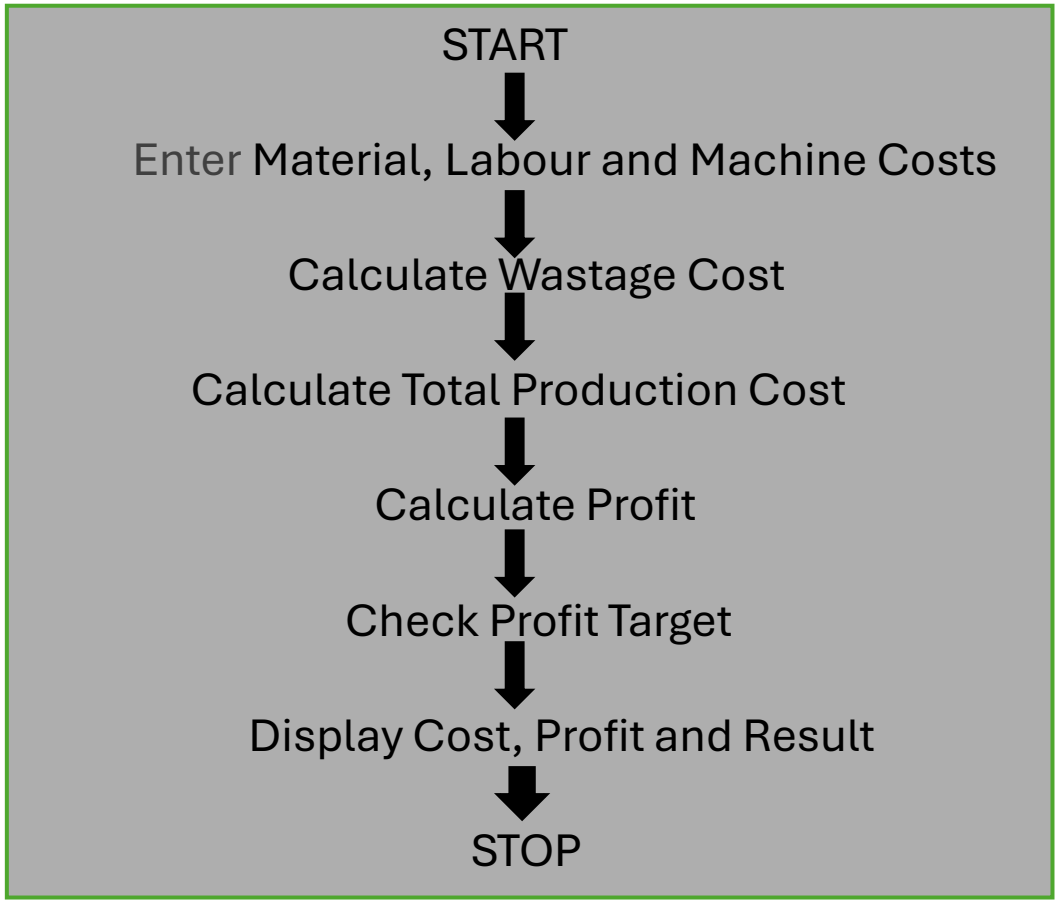
1. Material cost, labour charges, and machine costs are known.
2. Wastage is calculated based on a fixed percentage.
3. The selling price per product is given.
4. Total production cost includes all manufacturing expenses.
5. Profit is compared with the specified target to check profitability.

Case Study 07- Algorithm & Flowchart

ALGORITHM

Start.
Enter material cost, labour charges, and machine cost.
Calculate wastage cost.
Calculate total production cost.
Enter the selling price and calculate profit.
Check whether the profit meets the target.
Display total cost, profit, and result.
Stop.

FLOWCHART



Case Study 07 – pseudocode & python code

PSEUDOCODE

```
START
INPUT costs and selling price
CALCULATE total cost
CALCULATE profit

IF profit >= target
   PRINT "Target Achieved"
ELSE
   PRINT "Target Not Achieved"
END IF

STOP
```

PYTHON SOURCE CODE

```
material = 500
labour = 200
machine = 100
wastage = 50
selling_price = 1100
target = 200

total_cost = material + labour + machine + wastage
profit = selling_price - total_cost

print("Total Cost:", total_cost)
print("Profit:", profit)

if profit >= target:
   print("Target Achieved")
else:
   print("Target Not Achieved")
```

TEXT CASES WITH EXPECTED OUTPUT

Test Case	Material	Labour	Machine	Wastage	Selling Price	Target Profit	Expected Output
TC1	500	200	100	50	1100	200	Profit: 250 — Target Achieved
TC2	500	200	100	50	900	200	Profit: 50 — Target Not Achieved
TC3	500	200	100	50	1050	200	Profit: 200 — Target Achieved

OUTPUT SCREENSHOT



CONCLUSION

The program calculates the total manufacturing cost and profit, including material, labour, machine, and wastage costs. It checks whether the profit meets the target and displays the result.

Case Study 08- Multi-Currency Travel Budget Analysis

OBJECTIVES

To convert travel expenses using the given exchange rates.
To calculate currency conversion fees.
To compare total travel expenses with the planned budget.
To identify the largest travel expense category.
To determine whether the total expenses are within the budget.

ASSUMPTIONS

Travel expenses and exchange rates are given.
Currency conversion fees are fixed.
All expenses are converted into one common currency.
The total expenses are compared with the planned budget.
The largest expense category is identified from the expenses.

Case Study 08 – Algorithm & Flowchart

ALGORITHM

Start.
Enter travel expenses, exchange rates, and budget.
Convert expenses into a common currency.
Calculate conversion fees.
Calculate total expenses.
Compare total expenses with the budget.
Identify the largest expense category.
Display total expenses and budget status.
Stop.

FLOWCHART

START
↓
Enter Expenses, Rates, Budget
↓
Convert Currency
↓
Calculate Fees and Total
↓
Find Largest Expense
↓
Compare with Budget
↓
Display Result
↓
STOP

CASE STUDY 08 – PSEUDOCODE & PYTHON CODE

PSEUDOCODE

```
START
INPUT expenses, rates, budget
CONVERT expenses
CALCULATE total with fees
FIND largest expense
IF total <= budget
   PRINT "Within Budget"
ELSE
   PRINT "Over Budget"
STOP
```

PYTHON SOURCE CODE

```
expenses = [100, 200, 300]
rate = 1.0
fee = 10
budget = 600

total = sum(expenses) * rate + fee

print("Total:", total)
print("Largest Expense:", max(expenses))

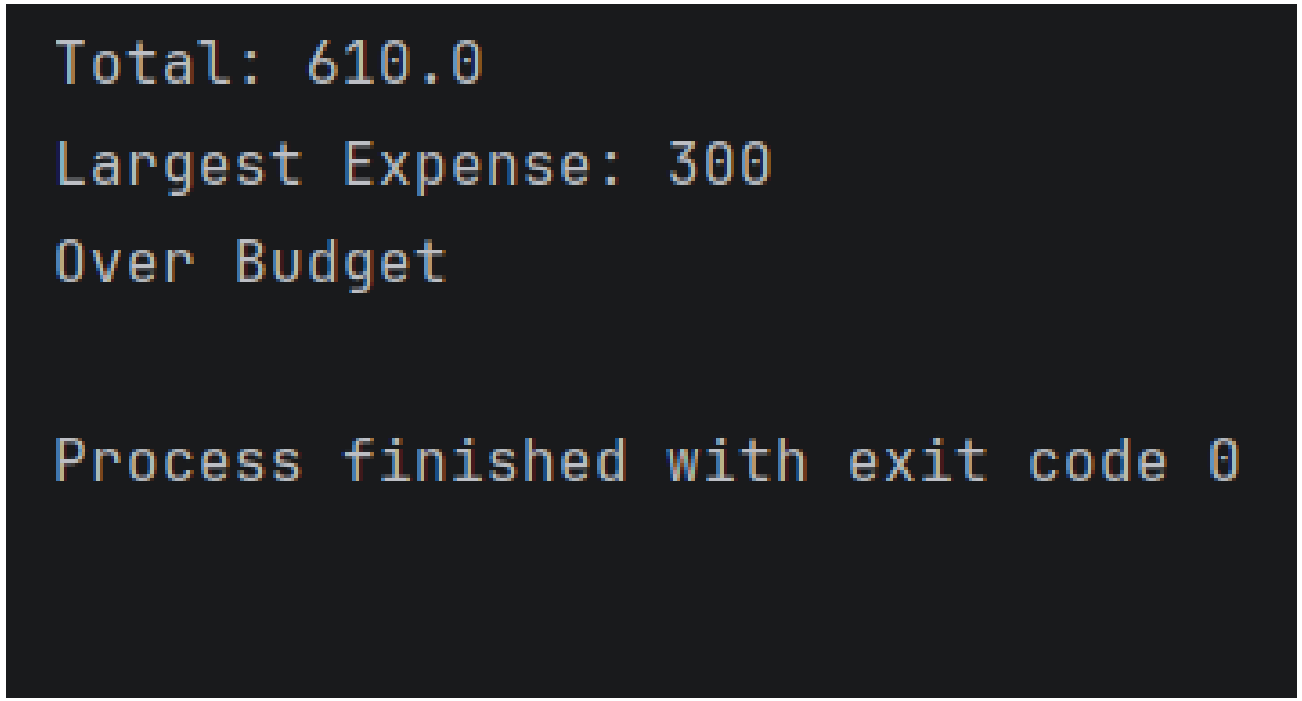
if total <= budget:
   print("Within Budget")
else:
   print("Over Budget")
```

CASE STUDY 08 – Testing & Conclusion

TEXT CASES WITH EXPECTED OUTPUT

Test Case	Expenses	Budget	Expected Output
TC1	100, 200, 300	600	Total: 610, Over Budget
TC2	100, 150, 200	500	Total: 460, Within Budget
TC3	50, 100, 150	400	Total: 310, Within Budget

OUTPUT SCREENSHOT



CONCLUSION

The program converts travel expenses, calculates the total cost, identifies the largest expense, and checks whether the expenses are within the budget.

CASE STUDY 09 - Campus Event Financial Planning

OBJECTIVES

To calculate venue, equipment, catering, transport, and certificate costs.
To calculate the total event cost.
To determine participant contributions.
To calculate the sponsorship shortfall.
To find the minimum number of registrations needed to break even.

ASSUMPTIONS

All event expenses are known.
Participant registration fees are fixed.
Sponsorship amount is given.
Each participant pays the same registration fee.
The break-even count is calculated based on total costs and income.

CASE STUDY 09 – Algorithm & Flowchart

ALGORITHM

Start.
Enter all event costs, registration fee, and sponsorship.
Calculate total event cost.
Calculate sponsorship shortfall.
Calculate participant contributions.
Find the minimum registrations to break even.
Display the results.
Stop.

FLOWCHART

START
↓
Enter Event Costs
↓
Calculate Total Cost
↓
Calculate Shortfall
↓
Find Break-even Registrations
↓
Display Results
↓
STOP

CASE STUDY 09 – PSEUDOCODE & PYTHON CODE

PSEUDOCODE

START
INPUT event costs, fee, sponsorship
CALCULATE total cost
CALCULATE sponsorship shortfall
CALCULATE participant contributions
FIND break-even registrations
DISPLAY results
STOP

PYTHON CODE

```
cost = 21000
sponsorship = 15000
fee = 200

shortfall = max(0, cost - sponsorship)
participants = (shortfall + fee - 1) // fee

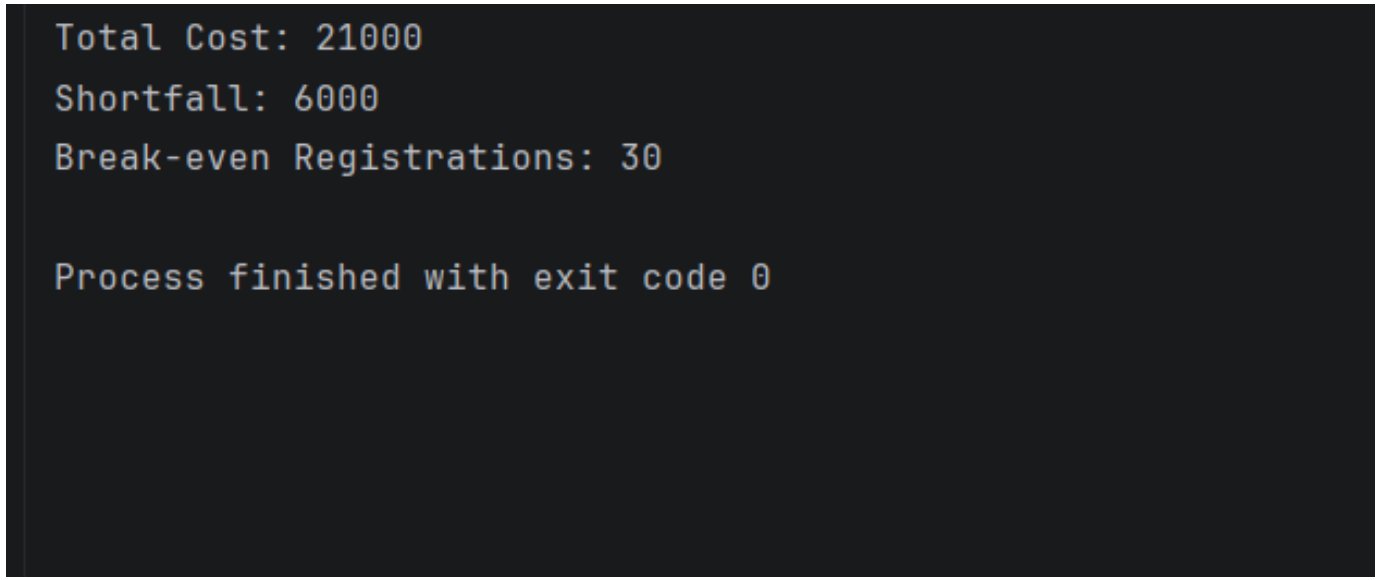
print("Total Cost:", cost)
print("Shortfall:", shortfall)
print("Break-even Registrations:", participants)
```

CASE STUDY 09 – Testing & Conclusion

TEST CASES WITH EXPECTED OUTPUT

Test Case	Total Cost	Sponsorship	Expected Shortfall	Registrations
TC1	21000	15000	6000	30
TC2	20000	16000	4000	20
TC3	15000	15000	0	0

OUTPUT SCREENSHOT



CONCLUSION

The program calculates the total event cost, sponsorship shortfall, and minimum number of registrations required to break even.

CASE STUDY 10 - Examination Result and Grace-Mark Validation

OBJECTIVES

To calculate total marks across multiple subjects.
To check subject-wise pass criteria.
To apply the specified grace-mark policy.
To ensure marks do not exceed maximum marks.
To generate the final result and classification.

ASSUMPTIONS

Marks for all subjects are provided.
Maximum marks for each subject are fixed.
Minimum pass marks are specified.
Grace marks are given according to the policy.
Final marks cannot exceed maximum marks.

CASE STUDY 10 – ALGORITHM & FLOWCHART

ALGORITHM

Start.
Enter marks for all subjects.
Check subject-wise pass marks.
Apply grace marks if eligible.
Ensure marks do not exceed maximum marks.
Calculate total marks and classification.
Display the final result.
Stop.

FLOWCHART

START
↓
Enter Marks
↓
Check Pass Criteria
↓
Apply Grace Marks
↓
Validate Maximum Marks
↓
Calculate Total & Grade
↓
Display Result
↓
STOP

CASE STUDY 10 – PSEUDOCODE & PYTHON CODE

PSEUDOCODE

START
INPUT subject marks
CHECK pass criteria
APPLY grace marks if eligible
LIMIT marks to maximum
CALCULATE total and grade
DISPLAY final result
STOP

PYTHON SORCE CODE

```
marks = [35, 32, 40]
grace = 3
passed = []

for m in marks:
   if m < 35 and m + grace >= 35:
      m = min(m + grace, 100)
   passed.append(m)

print("Marks:", passed)
print("Total:", sum(passed))
print("Result:", "Pass" if all(m >= 35 for m in
passed) else "Fail")
```

Case Study 10 – Testing & conclusion

TEST CASES WITH EXPECTED OUTPUT

Test Case	Input Marks	Expected Output
TC1	35, 32, 40	Total: 110, Pass
TC2	30, 25, 40	Total: 95, Fail
TC3	50, 60, 70	Total: 180, Pass

OUTPUT SCREENSHOT



CONCLUSION

The program checks subject-wise marks, applies grace marks when eligible, calculates total marks, and displays the final pass or fail result.